Leveraging Difference Recurrence Relations for High-Performance GPU Genome Alignment

Alberto Zeni
Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Italy
Italy
NVIDIA Corporation
USA
alberto.zeni@polimi.it

Seth Onken
NVIDIA Corporation
USA
sonken@nvidia.com

Marco Domenico Santambrogio
Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Italy
Italy
marco.santambrogio@polimi.it

Mehrzad Samadi
NVIDIA Corporation
USA
msamadi@nvidia.com

Abstract

Genome pairwise sequence alignment is one of the most computationally intensive workloads in many genomic pipelines, **often accounting for over 90% of the runtime of critical bioinformatics applications**. Recent advancements in sequencing technologies keep increasing the throughput of genomic sequencing data while decreasing the associated cost, emphasizing the need for fast and accurate software to perform sequence analysis, given the quadratic complexity of exact pairwise algorithms. In this challenging scenario, we present **the first fully GPU-accelerated version of the *KSW2* genome alignment library**. Results show that our high-performance implementation achieves up to **1145.17 Giga Cell Updates Per Second (GCUPS) and speedups up to 72.83× on a single NVIDIA Tesla H100 over the state-of-the-art baseline software running on two Intel Xeon Platinum 8358 processors** with a total of 128 CPU threads, while preserving alignment accuracy. Using the same configuration, **we demonstrate a 66.00× speedup, versus *ksw2d-fast*, a state-of-the-art improved version of one of the *KSW2* algorithms**. Furthermore, **we compare our implementation against a recently proposed FPGA implementation of *ksw2z*, achieving speedups up to 156.37× using a single H100 GPU.** To further highlight the impact of our work, **we integrate our accelerated kernels within one of the most used aligners and mappers in the State Of the Art, called *minimap2*, demonstrating runtime improvements by up to 8.51× and 8.03× using a single H100 GPU against the baseline software and *mm2-fast***, an optimized version of *minimap2* which integrates *ksw2d-fast* as its core aligner. Our design accelerates **all the algorithms** of the **state-of-the-art *KSW2* aligner suite (*splice, double- and single- gap affine*) and supports the Z-drop heuristic and banded alignment** as the original software to reduce the processing time further if needed. Finally, **we evaluate our application on the H100 GPU, adapting the Berkeley Roofline model for *KSW2* and demonstrating that our implementation is near optimal on our target GPU** architecture.

PACT '24, October 14–16, 2024, Long Beach, CA, USA

ACM ISBN 979-8-4007-0631-8/24/10
https://doi.org/10.1145/3656019.3676894

Keywords

GPU, Genomics, Genome Alignment, SIMD, DPX, KSW2, minimap2

ACM Reference Format:
Alberto Zeni, Seth Onken, Marco Domenico Santambrogio, and Mehrzad Samadi. 2024. Leveraging Difference Recurrence Relations for High-Performance GPU Genome Alignment. In *International Conference on Parallel Architectures and Compilation Techniques (PACT '24), October 14–16, 2024, Long Beach, CA, USA.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3656019.3676894

1 Introduction

Next Generation Sequencing (NGS) technologies are propelling incredible advancements across the medical and biological domains [21]. The capability to sequence a patient's genome, transcriptome, and proteome within a single day has accelerated the pace and precision of any diagnosis. In particular, *omics* data have emerged as a crucial asset in identifying DNA mutations linked to cancer, enhancing the understanding of human biology, and refining knowledge about inter-individual variations, paving the way for optimized personalized medicine [2, 17, 22, 25]. One key component in sequence analysis is the detection of similarities between two sequences via alignment. This methodology is one of the most commonly used workhorses for numerous applications, including correcting raw sequencer reads, assembling reads into whole genomes, and conducting searches within databases for sequences that bear resemblance. Most existing aligners exploit Dynamic Programming (DP), a strategy used for handling the sequence alignment problem in its various forms (e.g., global, semi-global, local) [3, 12, 14]. The Needleman–Wunsch (NW) [18] and Smith–Waterman (SW) [23] algorithms stand out as the most widely utilized examples of global and local DP-based alignment algorithms. However, they demand quadratic time complexity, leading many research efforts focusing on improving the performance of sequence alignment algorithms and their respective variations. The popular *KSW2* suite of genome aligners proposed by Li [14] presents multiple aligners based on the Suzuki–Kasahara Alignment (SKA) algorithm [26], employing difference recurrence relations to improve the efficiency and throughput of the alignment process without impacting accuracy. In particular, the *KSW2* suite

of aligners implements *spliced*, *single-*, *double-gap* affine aligners, together with additional features such as the Z-drop heuristic and banded alignment. Notably, the *KSW2* aligners are employed by *minimap2* [14], arguably the most popular software for genome analysis. *Minimap2* is a versatile tool for the alignment of genomic or protein sequences; it is mainly known for its efficiency in handling long reads' alignment, making it an essential tool in genomics research and one of the most used tools by hospitals and scientists in the State Of the Art. However, while significantly improving the throughput of genome analysis applications, the sole use of these highly optimized algorithms can still result in impractically long execution times, particularly when analyzing massive amounts of data [10]. Indeed, general purpose CPUs are often unable to deliver the computational power required by modern genomics applications [24]. For this reason, hardware accelerators are becoming increasingly popular to compute exact and heuristic algorithms [4, 7, 16, 19, 34, 35], thus significantly accelerating the sequence alignment process. In this context, Graphical Processing Units (GPUs) represent a valid alternative for general-purpose architectures. Moreover, GPUs have already proven to achieve outstanding results on genomics applications while requiring relatively low power consumption when compared against general-purpose Central Processing Units (CPUs) [4, 7, 8, 15, 16, 35]. Although numerous hardware-accelerated implementations of genome alignment algorithms already exist, a fully hardware-accelerated design of all *KSW2* aligners is notably missing in the State Of the Art. In this context, we propose the first fully GPU-accelerated version of *KSW2*. Results indicate that our high-performance GPU implementation, exploiting *Dynamic Programming X (DPX)* and *half2* arithmetic instructions, attains a maximum of 1145.17 Giga Cell Updates Per Second (GCUPS) on a single NVIDIA Tesla H100, achieving speedups of 72.83× with one GPU, in comparison to the state-of-the-art baseline software running on two Intel Xeon Platinum 8358 processors employing 128 CPU threads, while also maintaining the same accuracy as software. Using the same machine configuration, we observe a 66.00× speedup when compared to *ksw2d-fast* [11], an optimized version of the algorithm optimized by Kalikar et al. exploiting AVX512 vectorized instructions. We also compare our design with a recently proposed FPGA implementation of *ksw2z* [33], observing speedups of up to 156.37× when using a single H100 GPU. Furthermore, we integrate our application into one of the most widely utilized aligners and mappers in the State of the Art, namely, *minimap2*, and demonstrate that our implementation running on a single H100 GPU improves its runtime by up to 8.51× and, notably, showcases improvements of up to 8.03× against *mm2-fast* [11], Kalikar et al. optimized version of *minimap2* which leverages *ksw2d-fast* as its core aligner.

To summarize, our main contributions are:

- The first **high-performance**, **multi-GPU** implementation of the ***KSW2* alignment suite**, supporting ***spliced***, ***single-*** and ***double-gap*** affine **alignment**, together with supporting the Z-drop heuristic, banded computation and with **backtrace/cigar** generation support.
- An integration of our design within ***minimap2***, a state-of-the-art aligner, and mapper.
- An adaptation of the **Berkeley Roofline Model** to our design and underlying hardware demonstrating that performance is near optimal on the NVIDIA H100 GPU.

The remainder of the paper is organized as follows. Section 2 describes the original software algorithm we optimize for GPU execution, while Section 3 provides an overview of the related work to our design. Section 4 describes our implementation and optimizations. Section 5 presents the integration of our design within *minimap2* [14], Section 6 presents our experimental results, while Section 7 illustrates the Roofline model used to analyze our implementation. Section 8 summarizes our contributions and outlines future work.

2 Background

In this Section, we employ a bottom-up approach to describe the algorithm we accelerated on GPU. First, we focus on the the SKA algorithm [26] (Section 2.2), and the additional features proposed by Li [14] (Section 2.3) in *KSW2*. SKA is a semi-global alignment algorithm based on Smith–Waterman–Gotoh (SWG) (Section 2.1) [9, 23]. Finally, we briefly describe *minimap2*, highlighting its functionalities and bottlenecks.

2.1 The Smith-Waterman-Gotoh Algorithm

Given two input sequences $a = a_1a_2 \dots a_m$ and $b = b_1b_2 \dots b_n$ of length m and n the goal of the SWG algorithm [9, 23] is to find the highest-scoring *semi-global* alignment between a and b of the forms $a_1a_2 \dots a_i$ and $b_1, b_2 \dots b_j$, for some $i \leq m$ and $j \leq n$ that are chosen to maximize the score. To compute the similarity score between the two sequences, three DP matrices are defined, namely S, E, F, and a scoring matrix M. The computation of the three matrices is defined as follows:

$$
\begin{aligned}
S(i,j) &= \begin{cases} 0 & (i=0, j=0) \\ -G_{open_v} - j \cdot G_{ext_v} & (i \neq 0, j=0) \\ -G_{open_h} - j \cdot G_{ext_h} & (i=0, j \neq 0) \\ max \begin{cases} S(i-1,j-1) + M(a_i, b_j) \\ E(i-1,j) - G_{ext_h} \\ F(i,j-1) - G_{ext_v}, \end{cases} & (i \neq 0, j \neq 0) \end{cases} \\
E(i,j) &= \begin{cases} -G_{open_h} - i \cdot G_{ext_h} & (j=0) \\ -\infty & (i=0) \\ max \begin{cases} S(i,j) - G_{open_h} \\ E(i-1,j) - G_{ext_h}, \end{cases} & (i \neq 0, j \neq 0) \end{cases} \\
F(i,j) &= \begin{cases} -G_{open_v} - j \cdot G_{ext_v} & (i=0) \\ -\infty & (j=0) \\ max \begin{cases} S(i,j) - G_{open_v}) \\ F(i,j-1) - G_{ext_v}, \end{cases} & (i \neq 0, j \neq 0) \end{cases}
\end{aligned}
\tag{1}
$$

where G_{open} and G_{ext} are the gap opening and gap extension costs, respectively. The gap penalty function is expressed in linear form as $G(l) = G_{open} + l \cdot G_{ext}$. In the formulae, the vertical and horizontal gap penalties are distinguished by h and v subscripts, respectively, while they often might have the same value in literature applications. The origin cell $O(0,0)$ is initialized with 0, while the other edge cells are initialized as a contiguous gap from the origin. $M(a_i, b_j)$ is the value of M computed with the a and b characters.

2.2 Difference Recurrence Relations

In the original SWG algorithm, a comparison operation using an absolute value is embedded in every computation step, as every

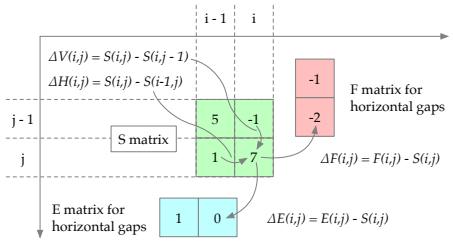
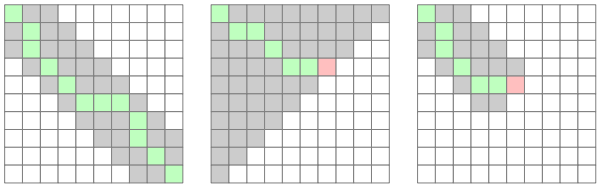


Figure 1: Graphical representation of the four difference matrices described in SKA. ΔH and ΔV represent the differences between horizontally and vertically adjacent cells in S. ΔE and ΔF are the differences between a pair of cells at the same i,j-coordinates in E and S, and F and S, respectively.



(a) Banded **(b) Z-dropped** **(c) Banded & Z-dropped**

Figure 2: Comparison between the search space of the multiple heuristics that are implemented in *KSW2*. Grey cells correspond to the computed cells by the algorithm, depending on the chosen heuristic, green cells showcase the optimal alignment path, and red cells indicate an interrupted-dropped alignment.

score in the S matrix is compared against 0. In contrast, semi-global and global alignment algorithms remove comparison operations with absolute values, enabling the formulation of the computation from recurrences into a *difference* form. This allows reconstructing the DP matrix S (Figure 1) from a collection of difference values between vertically or horizontally neighboring cells, plus the origin cell's initial value. Starting from the SWG formulation, the SKA algorithm defines four matrices storing the various difference (Δ) scores (ΔH, ΔV, ΔE and ΔF) among values of the previously defined DP matrices. These Δ values have specific range boundaries that can be computed from the gap penalty scores and the maximum absolute value in the scoring matrix M, ω. This characteristic allows the use of low-bit-count integers for the calculation and storage of the difference values. Leveraging difference recurrence relations enables the computation of the alignment using lower precision data types without sacrificing information from the original algorithm design, thereby ensuring that SKA achieves identical alignments to those generated by SWG. Furthermore, utilizing smaller integer precision data enables a faster computation of the various DP matrices and minimizes memory demands for their storage.

2.3 The KSW2 aligner suite

KSW2 exploits the SKA algorithm to implement three different aligners, *ksw2d*, *ksw2z* and *ksw2s*. While the overall structure of the *DP* matrix computation for the three algorithms remains the same, a small part of each aligner varies depending on the target workload. *ksw2d* aligns sequences utilizing distinct values for horizontal and vertical gaps, thus closely resembling the formulation of Equation (1), while in *ksw2z* the two gaps are considered having the same cost. The last aligner, *ksw2s*, performs the spliced alignment, where the core of the aligner remains the same as *ksw2z* while also accounting for an additional *junction* sequence, which enables the mapping of a sequence to multiple regions. One of the most used methodologies to reduce the search space in NW, SW, and SWG-like algorithms consists in computing only a fixed number of cells among the principal diagonal of the DP matrix. The key idea is that the optimal alignment of two sequences is likely to be found close to the main diagonal of the alignment matrix, especially if the two are known to be similar. Therefore, by focusing only on a fixed window of possible alignments around this main diagonal, the algorithm can save time and resources while still finding an alignment close to the optimal one. Another methodology for reducing the alignment time involves applying heuristics to the process [3, 13, 33–35]. The Z-drop heuristic, introduced first in BWA-MEM by Li [13], consists of halting the computation if the alignment score drops too fast in the diagonal direction. Indeed, in this case, the approach for the alignment is quite different from the above-mentioned banded method. In this scenario, the idea is to avoid searching for an optimal alignment if the two sequences differ too much from each other. In particular, this heuristic interrupts the alignment if there exist (i', j') and (i, j) where $i' < i$ and $j' < j$ such that:

$$S(i', j') - S(i, j) = Z + G_{ext} \cdot |(i - i') - (j - j')|, \qquad (2)$$

where Z is an arbitrary threshold. This heuristic is very similar to the one employed by X-drop [1, 3, 34], but it does not halt the alignment when encountering a single long gap. Figure 2 shows the different search spaces for the two methodologies; furthermore, it illustrates their combined behavior, as in the *KSW2* aligners.

2.4 Minimap2

Minimap2 is a versatile sequence alignment tool widely used in genomics and transcriptomics research [14]. It efficiently aligns DNA or RNA long reads to a reference genome or among them, allowing for accurate identification of sequence similarities and aligning reads to a reference. Furthermore, minimap2's speed, sensitivity, and ability to handle long reads make it an essential tool for tasks like genome assembly or transcriptome reconstruction. With its diverse range of applications, minimap2 provides researchers with a powerful tool to explore genomic data and gain deeper insights into the complex biological processes underlying diverse organisms. Minimap2 follows a typical seed-chain-align procedure as is used by most full-genome aligners [14]. Minimap2 creates an index of the reference genome or sequence database, creating a hash-map using k-mers, small sequences of length k, which minimap2 indicates as *minimizers*, exploiting them as keys to rapidly accessing the index. Common minimizers, or *anchors*, between the reference and target datasets are then exploited to individuate starting points for the chaining process. Anchors are then sorted depending on their position in the reference sequence and passed to the chaining

stage. This stage takes sorted anchors as input and extends them as co-linear ordered sub-sets of anchors, called *chains*, using a DP approach. Chains represent candidate regions where two sequences will most likely align. Minimap2 then employs bidirectional extension and DP to accurately align sequences, employing *KSW2* to perform this crucial step. To perform the actual alignment step, *minimap2* might utilize any of the three *KSW2* aligners (*ksw2d, ksw2s, ksw2z*) depending on the input dataset. Between them, *ksw2d* is the most frequent, providing the most biological information given its double-gap affine formulation.

3 Related Work

Most hardware acceleration efforts for pairwise alignment have focused on the SW, NW and SWG algorithms. These find exact alignments and have quadratic complexity in the lengths of the reads, as well as a relatively linear structure. Because of the most complex nature of the SKA algorithm implemented in *KSW2*, only part of its aligners have been implemented in hardware. In this section, we review the few efforts to accelerate the *KSW2* aligners suite, as well as the ones focusing on the acceleration of *minimap2*. Kalikar et al. [11] proposed an optimized implementation of *ksw2d*, which we indicate as *ksw2d-fast*, the double gap-affine aligner of *KSW2*, which achieves up to 2.2× speedup than the original software implementation. The authors were able to further parallelize the original implementation of *minimap2* by exploiting AVX-512 instructions in the chaining and alignment process. However, they only achieved half of the expected performance improvement with respect to the original SIMD implementation (using 128-bit vectorized instructions). Xu et al. [32] proposed an improved version minimap2 for NUMA multi-core CPUs, which exploited multiple NUMA-aware allocator policies to improve CPU resource utilization. However, they were only able to attain a 13% maximum improvement over the original software. Concerning hardware implementations, Teng et al. [27, 28] recently proposed two Field Programmable Gate Array (FPGA) designs of *ksw2z*, the single gap-affine implementation. In the first one, the authors implemented only a partial step of the *ksw2z* algorithm on a AMD Zynq Ultra96 FPGA, showing significant performance improvements, but only in simulation. Notably, they only implemented a very small section of the algorithm on FPGA, ignoring all the interconnection latencies. More recently, in their second work, the authors adapted GACT-X [29] to run a similar algorithm to the one proposed in *ksw2z*. Indeed, they were not able to reproduce the *ksw2z* entirely but rather a simpler version of the algorithm, thus not being able to match the algorithm's capabilities and accuracy. With this implementation, the authors achieved a 2× improvement concerning the software version of *ksw2z* when running on an Amazon F1 FPGA Instance, with a VU9P FPGA from AMD, against Intel Xeon E5-2658 with only 16 threads. An openly available FPGA implementation of *ksw2z* has recently become available in the literature [33]. The proposed design, run on an AMD Alveo U250, showed a 7.7× improvement against the baseline software implementation of *ksw2z* run against an Intel Xeon Platinum 8167M. While the proposed implementation shows significant performance benefits, it only implements the computation of the alignment score, omitting the computation of the backtrace step. Indeed, this step does not represent a computational bottleneck during the alignment process, but it is a fundamental step for alignment analysis. Notably, removing the backtrace step significantly simplifies the design of the architecture, as fewer resources need to be employed to perform each alignment. Concerning efforts to accelerate *minimap2*, a recent work by Sadasivan et al. [20], presented *mm2-ax*, an implementation of *minimap2* that accelerates its chaining step using GPUs. While observing significant speedups over *minimap2-fast*, Kalikar et al.'s optimization of *minimap2* with the *ksw2d-fast* algorithm, this work only focuses on specific workloads where the *KSW2* aligner load is negligible, thus not accounting for the aligner bottleneck. Notably, we can observe that no complete accelerated solution of the *KSW2* is present in the literature. Furthermore, the proposed efforts are significantly limited, both in terms of performance and flexibility. To the best of our knowledge, our work is the first and only attempt in the literature to propose a **high-performance multi-GPU implementation** of the ***entire KSW2* suite of aligners**. Our work outperforms all the designs present in the State Of the Art, both in terms of performance and flexibility, providing all the three aligners of ***KSW2* (*ksw2d, ksw2s, ksw2z*)**, supporting all their inputs and heuristics.

4 Implementation

The fundamental aspect of our implementation is employing as many parallelism levels as possible on the GPU. Intra-Sequence parallelism is achieved by exploiting warp-level intrinsics and 2-way Single Instruction Multiple Data (SIMD)-like computation of the various difference recurrence relations within the same anti-diagonal leveraging *DPX* and *half2* instructions. To achieve inter-sequence parallelism, we implement the parallel execution of multiple alignments by assigning each GPU block to an alignment. We attain multi-GPU parallelism by implementing a load balancer module that automatically divides the load of the various alignments to the available GPUs. In this section, we describe the various optimizations and design choices we implemented in our application. We base our design on the SKA algorithm [26] implemented by Li in the *KSW2* aligner suite [14]. In this work, we focus on implementing the spliced, single- and double-gap affine alignment algorithms, targeting the computation of the *difference matrices* for sequence alignment and their corresponding backtrace step. Even though the cigar computation does not represent a bottleneck for the algorithms, we still fully implemented it on GPU, as it provides crucial biological information of the alignment result. Furthermore, not computing it on GPU would require someone who is interested in knowing its result to re-execute the alignment in software, thus losing the performance benefits of a GPU-accelerated implementation. Our implementation retains all the features supported by the baseline software implementation of *KSW2* (Section 2.3), making it a drop-in replacement for every application that uses it as its core aligner. For the sake of simplicity, we refer our optimizations to the *ksw2d* aligner, the most complex out of the three in *KSW2*, as all the applied optimizations are employed in all three aligners.

4.1 Intra-Sequence Parallelism

We first consider intra-sequences parallelism, which is the parallelization of a single sequence alignment and its anti-diagonals computation. The computation of difference recurring relations

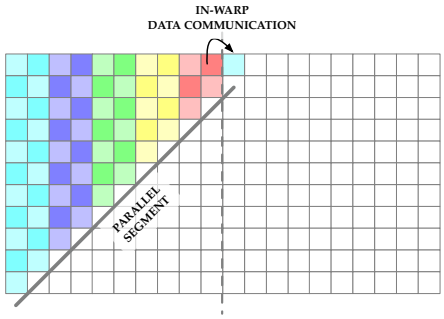


Figure 3: Each anti-diagonal is divided into segments (dotted line); a thread computes two different recurrence relations in a SIMD fashion through *DPX* or *half2* instructions on the H100 and A100, respectively. In our configuration, the segment length equals 64 cells (32 threads per warp × 2 cells computed per thread). We exploit in-warp register communication to pass the data regarding neighboring cells when computing multiple segments.

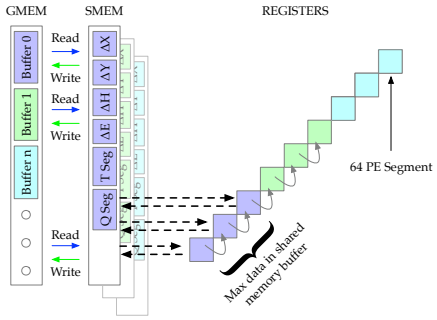


Figure 4: Double buffering mechanism for each anti-diagonal. Anti-diagonal data is stored linearly in global memory; we load/write partial data of an anti-diagonal on shared memory. Each shared memory buffer is read/written by a segment of PEs (indicated as a single block in the anti-diagonal), which exploits registers and interacts with the shared memory only if needed (black dotted arrows); otherwise, it shares data with other threads using in-warp instructions (gray arrows).

follows the same wavefront parallel pattern as NW and SW algorithms, computing the anti-diagonals of the alignment matrix in parallel. In fact, as the computation of the neighboring cells of an anti-diagonal only depends on previous algorithm iterations, we can leverage their independence to calculate their value in parallel. The baseline CPU implementation of *KSW2* [14] exploits CPU SIMD vectorized instructions and the wavefront pattern to compute 16 cells in parallel per iteration in the DP matrix. To improve upon this, increasing the degree of parallelism of each anti-diagonal update, we optimize the code to make each GPU thread compute 2-way SIMD instructions. We leverage the CUDA instruction set *DPX* and *half2* APIs to enable vectorized instructions at the thread level on the GPU, significantly increasing the amount of data we can compute per iteration while still keeping the required information per cell at the same bit count as the CPU version. *DPX* instructions can be hardware accelerated only on Hopper GPUs and are executed on custom integer compute units available only on this line of NVIDIA GPUs, while *half2* instructions employ the utilization of floating point compute units and can be hardware accelerated on NVIDIA GPUs from the Volta generation onwards. Despite being fundamentally different in the way they are executed on the GPU, the model these two instructions follow is basically the same. Indeed, to exploit these particular instructions, we have to organize data on the GPU kernel to contain two different 16 bit variables into a single 32 *packet* variable. Then, CUDA APIs expose multiple dynamic programming and basic functions that interpret the 32 *packet* variable as two different lanes while executing the same instruction for both lanes, essentially executing a SIMD instruction on each GPU thread. To optimize our resource utilization, we schedule 32 threads per block. This enables the computation of a total of 64 cells in parallel, thus achieving a high level of parallelism while minimizing the number of wasted resources. In fact, if the number of threads exceeds the length of an anti-diagonal, many of the threads will indeed stall, decreasing the performance of our kernel. Additionally, having scheduled a single thread-warp per block, we enable in-warp thread communication when computing each anti-diagonal. Since the amount of resources per thread can be stored locally and communicated to the other threads via in-warp instructions, we can only use registers to store intermediate values in our kernel computation, removing the need to allocate additional shared or global memory for every alignment. These strategies enable us to rapidly and efficiently compute the various cell updates and the maximum value of the alignment (which is used as a starting point when computing the backtrace). To compute an anti-diagonal update in parallel, we split each anti-diagonal into multiple *segments*, as shown in Figure 3, as our compute window is composed by a total of 64 Processing Element (PE). The segment length is equal to our processing window of 64 cells, and once we compute a segment, the kernel initiates the computation of the subsequent segment automatically. This process is iterated until the entire anti-diagonal is computed. During the computation, every thread loads a register of 32-bits containing the information of two neighboring cells (one per PE) of the previous anti-diagonal for every matrix needed for the computation of the alignment. In a similar manner, we load two different bases per sequence while also computing the matching values through the *scoring matrix M*, enabling generic scoring. If required, we also reduce the width of the anti-diagonal, matching the value indicated by the user enabling banded alignment computation. Using the information relative to the previous anti-diagonal contained in the registers, the kernel computes the difference in the relations between the current anti-diagonal. Our algorithm leverages in-warp thread communication to compare the values inside the cells with the newly computed ones. Threads in the same warp can communicate through registers using CUDA in-warp primitives instead of passing through global or shared memory, which maximizes communication speed. To perform operations in the context of the two variables stored in a register, we employ SIMD vectorized 2-way instructions to improve performance further. During this process, we also save the direction of the dependency we utilized in computing a cell to

be later used inside the backtrace step. When an anti-diagonal has been fully computed, our kernel reconstructs the scoring values using the difference relations valued just computed and proceeds to check for early termination according to the Z-drop heuristic. Given that all the $KSW2$ algorithms exploit different recurrence relations, we were able to store a very limited quantity of data on the device for each alignment. Indeed, because we only need to compute the different relations between the DP matrices to perform the alignment, we only store a single anti-diagonal per difference relation matrix plus one for the scoring matrix S. Furthermore, to limit the amount of latency of the algorithm, we implement a double buffering mechanism to maintain in shared memory the majority of resources required by the algorithm. Thanks to the limited amount of resources the algorithm requires, we can maintain a window of up to 512 cells in shared memory per block, containing the correspondent query and target sequences and all the various correspondent recurrence relations. When computing the alignment, our kernel loads the relative information for a 64 PE segment from a shared memory buffer and maintains it in registers for the entire segment computation to benefit from in-warp register instructions (Figure 4). When our *PE* segment is required to load data beyond the limit of the loaded buffer, we commit the various results to global memory and load data regarding the following cell window from global memory. Doing so enables us to exploit memory burst by reading segments of continuous memory on the off-chip GPU memory, reducing overall latency, increasing data locality, and significantly improving performance. Once the *DP* matrix has been completely calculated, we compute the backtrace of the alignment, creating a *cigar* string (section 4.4). We then send the *cigar* information to the host together with its alignment score.

4.2 Inter-Sequence Parallelism

Intra-sequence parallelism improves the alignment computation for a single sequence pair but does not fully utilize the computational resources of the GPU architecture. To harness the GPU's computational capabilities more effectively, our application is designed to concurrently align numerous pairs of sequences by allocating each alignment to a GPU block, thereby exploiting inter-sequence parallelism. The scheduling of GPU blocks is based on the required number of alignments. In our experiments, we used two different GPUs, NVIDIA A100 and the NVIDIA H100 data center GPUs. These GPUs have 192KB/Streaming Multiprocessor (SM) and 256KB/SM of shared memory available, respectively. Although a maximum of six vectors must be stored (depending on the chosen $KSW2$ alignment algorithm), implying they might be stored in shared memory on the GPU, this is often not feasible in practice. Indeed, despite the capability to allocate 255KB of memory per block on the NVIDIA H100, the device possesses only 256KB of memory per SM; the same concept can be applied to the NVIDIA A100. Since each SM on the device can run up to 32 blocks concurrently, if excessive memory is reserved by a single block, an SM has to interchange data with the GPU's Global Memory (GMEM) for each block computation or the GPU has to limit the number of concurrently executed blocks to an amount lower than 32, significantly diminishing the possible attainable performance. To achieve optimal board-level

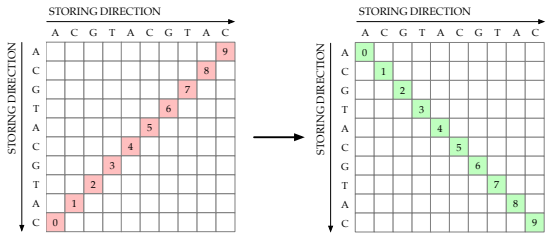


Figure 5: The vertical sequence of every sequence pair is reversed to exploit coalesced memory access on the GPU. Numbers inside the cells indicate the computation order of the cells, as well as the data access order of the sequences. By reversing one of the sequences, the computational pattern and access pattern of the sequences coincide, thus we can exploit memory burst to attain significantly faster accesses.

utilization, it is necessary to compute several blocks per SM in parallel. Therefore, to surpass the restrictions we listed above and avoid shared memory contention, we size our cache buffers utilized in our double-buffering mechanism according to the available resources on the board we are targeting. This operation is done automatically and enables us to maintain the maximum flexibility for the compatibility of our kernel on multiple boards. With respect to our target boards, we attain a buffer of 384 and 512 cells on the NVIDIA A100 and H100, respectively. Since dependency information for the alignment backtrace step is not accessed as often as the other vectors, we decided to store this information on the GPUs' High Bandwidth Memory (HBM). Indeed, both our target boards offer very high memory bandwidth and relatively low latency off-chip HBM; thus, we were able to still achieve peak performance using this configuration.

4.3 Host Optimizations

To make our kernel processing more efficient on the GPU, we introduce multiple CPU host optimizations. First, we allocate host vectors using pinned memory to optimize data movements and enable stream computation overlapping. Our design implements multiple GPU streams to hide some of the computation required by the software host. We schedule a total of 16 streams per GPU and divide the target GPU resources accordingly. Each GPU stream automatically starts its execution when its resources have been fully allocated; this process is performed by assigning multiple alignments to the same stream. For all the alignments we need to perform, we evaluate the necessary resources on GPU using the length of the sequences we want to align as the unit of measure. After an alignment has been evaluated, the host either assigns it to the current stream or schedules the current stream for execution and assigns the alignment to the next available stream. We iterate this process until all the streams have been filled. The host is then in charge of retrieving the results from the GPU. It starts by collecting the data from the first executed stream, and after collecting its data, it proceeds with the execution if more alignments need to be performed. By splitting the computation into multiple streams, we are able to increase the degree of parallelism of our implementation, being able to hide part of the computation required

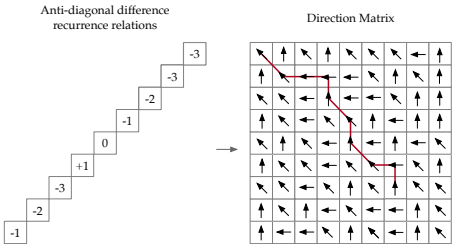


Figure 6: Comparison of space requirement for the scoring of the anti-diagonal difference relation and the Direction Matrix. Scoring related data requires only the space of an anti-diagonal, while to attain the backtrace of a matrix (shown in red), we need the entire matrix of directions, which we store using 8-bit variables.

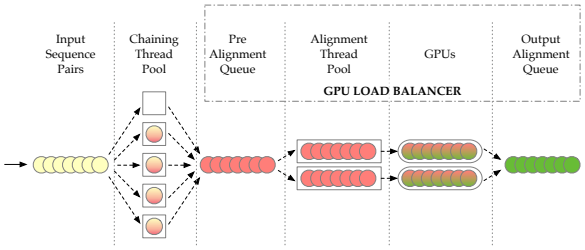


Figure 7: Overview of our modified *minimap2* implementation. A thread pool parallelizes the computation of the chaining step as in the original software, while we batch alignment tasks in a single queue. Our *load balancer* organizes inputs for the various GPUs in the system, ensuring that all GPUs are always loaded with a batch of alignments.

for pre-processing the inputs before sending them to the GPU. Indeed, we pre-process the input sequences to speed up the alignment on GPU. Normally, when aligning two sequences, one of the two has to be accessed backward, resulting in memory performance degradation since characters are read in the opposite direction of the memory. To ensure coalesced data access and exploit memory burst, we reverse one of the two sequences for each given pair, as shown in Figure 5. This optimization linearizes the GPU memory data access, increasing performance while still preserving the correct solution.

4.4 Backtrace

While the previous sections describe the most computationally intensive part of the alignment process, applying those steps alone would not produce the information regarding the sequences' alignment, but rather only a score indicating their similarity, resulting in close to no information in the biological sense. Indeed, to attain the information regarding the actual alignment of the sequences, an additional step needs to be performed, called *backtracing*. To enable this feature, the alignment algorithms have to store the alignment score and the information about what caused that score to be chosen. In our algorithm, this information is expressed by storing the information regarding which dependency has been utilized to compute the score of each cell by saving the direction of the dependency (e.g., north, west, or northwest). To do so, we reserve additional space on the GPU global memory to store the above-mentioned information every time we compute a difference recurrence relation in a new matrix called *direction matrix* (Figure 6). Once the scoring algorithm has terminated, we can proceed to compute the *backtracing* process. We start from the cell of the *direction matrix* corresponding to the best similarity value (i.e., the alignment score) and move backward following the dependencies of the *direction matrix*. The path taken during the backtrace directly corresponds to the alignment of the sequences. Matches/mismatches come from diagonal moves, while gaps are introduced for vertical or horizontal moves. The final result is a complete alignment of the sequences, which may include several matches, mismatches, and gaps, depending on the sequences' similarity and the chosen scoring scheme. The output of the *backtracing* step is a sequence, which represents the correspondence of the two sequences, called *cigar*. To perform this operation on GPU, we assign a single thread at the end of the alignment step to reconstruct the cigar from the last operation of the alignment to the first one. It is important to note that while the *backtracing* operation is not particularly intensive (accounting for only 5% of the alignment runtime) and cannot be parallelized, it is crucial to have the support for performing this operation on the device itself. Omitting this operation significantly simplifies the kernel design by not having to account for the additional space complexity of the *direction matrix* and reducing the latency of the kernel, but renders the kernel useless for real-world applications. Indeed, without the biological information of the alignment, the kernel itself has no use for sequence analysis applications (e.g., drug discovery, cancer diagnosis). Furthermore, if someone requires the information regarding the alignment *cigar* after executing a kernel without the backtrace, the only way to attain that information would be to re-run the same alignment on CPU using the original software, making the accelerated implementation of the alignment practically useless.

4.5 Implementation with Multiple GPU Devices

To effectively exploit all the available multiple GPU resources in a given system, we implement a *load balancer* module (Figure 7-right). This optimization allows our design to run on multiple configurations, adapting the load on the number of GPUs on a given system. We parallelize the scheduling by exploiting a thread pool and assigning a CPU thread per GPU (*device-thread*). Each thread follows the tasks for streaming operation overlapping and input pre-processing as described in Section 4.3. When using multiple GPUs, alignments are organized in a single queue and subsequently pulled by each *device-thread* to fill the various devices. While loading the various GPU streams happens in a *round-robin* fashion, our balancer acts dynamically depending on the GPU load. Indeed, once a GPU has completed its execution, its corresponding *device-thread* proceeds to re-fill it with a new batch of alignments (regardless of the order in which its execution was scheduled to start), ensuring that no GPU remains idle while there is still work to be completed. In this way, we constantly maintain all the GPU devices active and ensure that alignments are distributed evenly, regardless of the number of GPUs or alignments we have to schedule. Additionally, since

Table 1: Experimental setup.

GPU/FPGA Model	Memory type	Memory capacity	CPU Model	RAM capacity
NVIDIA H100	HBM2e	80 GB	2 x Intel Xeon Platinum 8358	1 TB
NVIDIA A100	HBM2e	80 GB	2 x Intel Xeon Platinum 8358	1 TB
AMD ALVEO U250	DDR4	32 GB	Intel Xeon Gold 6234	128 GB

each thread operates independently of the others, our design also supports configurations with mismatched GPUs, increasing the flexibility of our implementation. Our *load balancer* module is also in charge of re-organizing the alignments after they have been executed on the various devices. To perform this operation, we assign a unique *idenfier* to every batch of scheduled alignments and task each *device-thread* to push the attained results into a queue after the alignments of a GPU have finished computing. Once all the alignments have been processed, the *load balancer* schedules a single thread to pull the finished results from the results queue and organize them based on their corresponding *idenfier*.

5 minimap2 Integration

To demonstrate the impact of the work, we integrate our accelerated implementation of *KSW2* into *minimap2* [14]. The baseline design of minimap2 exploits multiple threads to first perform the chain and then execute the alignment for the input long-read pairs. In particular, minimap2 assigns a *pair* of sequences per thread and proceeds to perform the various steps of chaining and aligning sequentially. While effective for parallelizing and distributing the process on the CPU since each thread operates on a pair of sequences, this procedure is not optimal with respect to our implementation. Indeed, simply replacing the software implementation of *KSW2* within minimap2 would have led to sub-optimal results, as each thread would require the alignment of a small batch of sequences every time it needs to call for *KSW2*, therefore limiting the amount of load the GPU could process in parallel. To surpass this limitation, we modify minimap2 to create batches of alignments to be sent to the GPU, as shown in Figure 7. We schedule a thread pool to compute the chaining step of the alignments with multiple threads, thus maintaining parallelism in the chaining step of minimap2 like the original software, but cache the multiple alignment requests in a single queue. The queue of alignments is then processed by another thread pool, which schedules a number of threads equal to the GPU devices present in the system, following the logic explained in Section 4.5. Our optimized minimap2 implementation, together with the integration of our optimized *KSW2* kernels, produces equivalent results as the original software version of minimap2 and enables us to significantly increase GPU utilization, showcasing over a 4.13× and 12.38× end-to-end runtime improvement when running using 1 and 4 GPUs, respectively, over minimap2 with only our GPU *KSW2* kernels integrated.

6 Discussion

In this section, we describe the experimental settings used to evaluate our methodology and present our performance results.

6.1 Experimental Setting

We first compare our *KSW2* implementation against the CPU-based algorithm as implemented in the original *KSW2* aligner suite [14]. Next, we evaluate *KSW2* against two accelerated versions of *KSW2*: Kalikar et al. optimized version of the double gap-affine aligner *ksw2d-fast* [11] and the current state-of-the-art FPGA implementation of the single gap-affined aligner, *ksw2z* [33]. Finally, we integrate our design into *minimap2* to show its benefit in a real-world state-of-the-art application and compare it against the original software and Kalikar et al. optimized implementation of minimap2 called *mm2-fast* [11]. Table 1 summarizes the platforms we used to collect our results. To test our design we randomly select 1M sequence pairs from two state-of-the art [6, 30] openly available PacBio Hi-Fi E.Coli[1] and C.elegans[2] datasets with an average sequence length of $12.92K$ and $5.79K$ characters respectively, which represent real-world use cases of minimap2. The results were collected on a dual-socket server with two Intel Xeon Platinum 8358 processors, 4 NVIDIA Tesla H100 PCIe GPUs (80 GB HBM2e), and 4 NVIDIA Tesla A100 PCIe GPUs (80 GB HBM2e) with 1024 GB of RAM. Each processor has 32 compute cores with two threads per core, totaling 128 threads. To compare our design against the recently proposed *ksw2z* FPGA design [33], we only use the same 1M randomly selected sequence pairs from the E.Coli[1] dataset, as the FPGA design does not support sequences longer than 8192 characters without modifying the kernel design. We collect the data of the FPGA platform on a server with 128 GB of RAM and an AMD Alveo U250 FPGA. For the comparison employing our design integrated within *minimap2* [14], we use the full E.Coli[1] and C.Elegans[2] datasets, accounting for a total of 1.8M and 7.5M alignments respectively. We compile all the state-of-the-art software with O3 optimizations and our design with CUDA 12.2. We used version 2.22 of *minimap2* for our integration and comparison with *mm2-fast* which is implemented on the same release of the original *minimap2* software. The reported runtimes for the alignment step are averaged over 50 executions across the various implementations, while end-to-end runtimes for Minimap2 are averaged over 10 executions due to the longer execution times, standard deviation across different runs remained below 2% in both cases.

6.2 Results

Table 2 summarizes the execution times of our design compared against all *KSW2* software run on 128 threads on two Intel Xeon Platinum Processors and FPGA [33] implementation. All the reported execution times were measured by running all *KSW2* aligners with the backtrace computation enabled. Additionally, to provide a fair comparison with software, we measure the PCIe communication time, and all the pre-processing needed for our implementation. *ksw2s* spliced alignment algorithm does not support banded alignment, both in software and GPU; thus, we analyze it when running without the bandwidth heuristic. Kalikar et al. optimized *ksw2d* supports both AVX2 and AVX512 instructions, improving upon the original software AVX 128-bit vectorized instructions Because of this, we report the execution times of all three of its implementations. Observe that our design achieves speedups

[1]https://www.ncbi.nlm.nih.gov/sra/?term=SRR11434954
[2]https://www.ncbi.nlm.nih.gov/sra/?term=SRR20766636

Table 2: Performance and execution times of all *KSW2* software and hardware-accelerated implementations on 1M sequences from *E.Coli*[1] and *C.Elegans*[2] datasets. *Z*-Drop and Banded heuristics have been set to 128 when enabled.

Implementation	Algorithm	Dataset	Full	Bandwidth	*Z*-Drop	*Z*-Drop & BW
			Runtime (s) - GCUPS			
Ours - A100	*ksw2d*	E. Coli	45.74 - 604.82	2.18 - 12.62K	4.93 - 5.61K	2.14 - 12.92K
		C. Elegans	203.82 - 606.40	9.55 - 12.89K	21.59 - 5.70K	9.49 - 11.09K
	ksw2z	E. Coli	44.18 - 626.17	2.08 - 13.23K	3.89 - 7.10K	2.12 - 13.03K
		C. Elegans	195.98 - 628.20	9.28 - 13.25K	17.26 - 7.13K	9.32 - 13.20K
	ksw2s	E. Coli	43.92 - 629.86	-	4.09 - 6.75K	-
		C. Elegans	193.03 - 637.81	-	17.94 - 6.85K	-
Ours - H100	*ksw2d*	E. Coli	24.33 - 1.13K	1.14 - 24.08K	2.63 - 10.48K	1.14 - 24.09K
		C. Elegans	112.57 - 1.09K	5.27 - 23.32K	9.66 - 10.32K	5.31 - 23.18K
	ksw2z	E. Coli	24.15 - 11.45K	1.15 - 24.04K	2.16 - 12.77K	1.14 - 24.09K
		C. Elegans	110.17 - 1.11K	5.20 - 23.67K	11.92 - 12.74K	5.18 - 23.72K
	ksw2s	E. Coli	0.91 - 11.44K	-	0.086 - 12.14K	-
		C. Elegans	110.31 - 1.11K	-	10.25 - 12.07K	-
Li [14]	*ksw2d*	E. Coli	670.12 - 41.28	83.67 - 330.64	80.60 - 343.23	83.20 - 332.49
		C. Elegans	2977.89 - 41.34	374.55 - 328.70	359.70 - 342.27	370.52 - 332.277
	ksw2z	E. Coli	652.38 - 42.40	78.95 - 350.41	75.84 - 352.98	78.37 - 352.98
		C. Elegans	2852.72 - 43.15	353.92 - 347.86	335.86 - 366.56	345.11 - 356.74
	ksw2s	E. Coli	695.98 - 39.75	-	80.72 - 342.73	-
		C. Elegans	3080.92 - 39.96	-	359.94 - 342.04	-
Kalikar et al. [11]	*ksw2d*	E. Coli	606.56 - 45.61	75.72 - 365.35	73.08 - 378.54	77.30 - 357.87
		C. Elegans	2709.68 - 45.43	341.52 - 360.49	327.51 - 375.91	349.63 - 352.12
Zeni et al. [33]	*ksw2z*	E. Coli	308.42 - 89.69	179.93 - 153.75	109.75 - 252.06	110.28 - 250.85

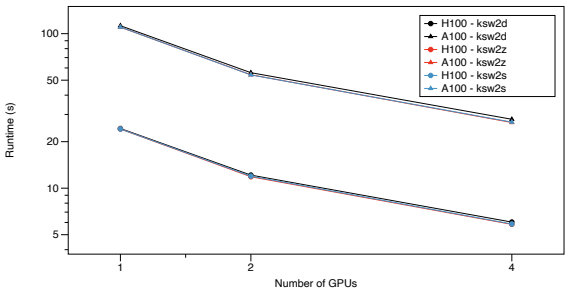


Figure 8: Overview of the runtimes (log-scale) of our implementation when scaling with Multiple GPUs aligning 1M sequence pairs from the *E.Coli*[1] Dataset.

Table 3: Analysis of *minimap2* [14] and *mm2-fast* [11] against our *minimap2* implementation with out *KSW2* GPU-accelerated design using the *E. Coli*[1] and *C. Elegans*[2] datasets.

Platform	Dataset	Total Runtime (s)	Alignment	Chaining	Seeding
			Runtime Percentage		
Ours - H100	E. Coli	1643.77	10.96%	50.46%	38.58%
	C. Elegans	2388.011	29.54%	36.71%	33.75%
Li [14]	E. Coli	6426.29	77.23%	13.23%	9.54%
	C. Elegans	20323.04	91.74%	4.41%	3.85%
Kalikar et al. [11]	E. Coli	6041.24	75.77%	13.77%	10.46%
	C. Elegans	19185.35	91.24%	4.77%	3.99%

ranging from 25.89× to 72.83× using a single NVIDIA Tesla H100, over the state-of-the-art baseline software running on two Intel Xeon Platinum 8358 Processors processors using 128 CPU threads, with equivalent accuracy. Using the same input settings, we demonstrate a speedups ranging from 24.41× to 66.00× on one H100 GPU versus the best performing implementation of *ksw2d-fast*, run on AVX512 vectorized instructions. We also observe significant improvements when comparing software against our design running on A100 GPUs, attaining up to 39.21× speedups against the baseline code, and up to 36.90× against *ksw2d-fast* using one NVIDIA A100 GPU. Notably, our implementation achieves peak performance of 1145.17 GCUPS when running the full alignment *ksw2z* algorithm, with similar performance on *ksw2s* and *ksw2d*. Furthermore, we achieve better performance improvements when applying the various heuristics (*Z*-drop and banded alignment) with respect to the CPU implementation. We also note that our multiple GPU implementation scales almost linearly in all the various configurations (Figure 8), this was achieved thanks to the concurrent execution of multiple GPU streams and our *load balancer* module. This means that our design is able to efficiently hide the time used for preprocessing the input sequences and data transmission over PCIe and is also able to effectively distribute the load on multiple GPU devices. When comparing our design against FPGA *ksw2z* [33], we observe speedups from 12.76× up to 156.37× using one NVIDIA H100 GPU against *ksw2z* running on FPGA. Furthermore, we notice that the FPGA implementation poorly scales when aligning the algorithms using heuristics methods, thus further increasing the speedup of our implementation in those situations. Finally, we analyze the improvements of *minimap2* [14] using our GPU-accelerated design in place of the original *KSW2* implementation (Table 3). Our design improves the performance throughput and latency of *minimap2* and *mm2-fast*, achieving speedups up to a factor of 8.51× and 8.03×, respectively. Notably, while *mm2-fast*

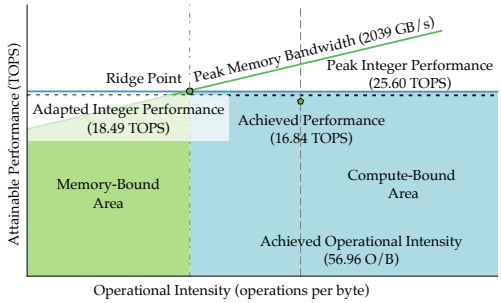


Figure 9: Roofline analysis for our kernel on the NVIDIA Tesla H100 GPU performing $1M$ alignments of the E.Coli[1] dataset running our optimized *ksw2d* aligner.

represents an improvement against the original implementation, we observe that its improvement remains marginal, especially when analyzing alignment-heavy datasets, which are one of the most common use cases for minimap2 [36].

7 Roofline Analysis

In this Section, we provide a detailed analysis of the optimized *KSW2* GPU design performance by exploiting the Berkeley Roofline model [5, 31] to model our specific implementation. The visually intuitive Roofline model utilizes a *bound and bottleneck* analysis approach to aid in comprehending the performance of a specific kernel. It outlines the various factors that affect computer system performance and correlates processor performance with off-chip memory traffic. By characterizing a kernel's performance in Tera Operations Per Second (TOPS) as a function of its Operational Intensity (OI) on a 2D log-log scale graph, the Roofline model incorporates integer performance, operational intensity, and memory bandwidth. To achieve this, the model employs the X-axis to represent OI, which measures instructions per byte of GMEM traffic, and the Y-axis to represent TOPS, given that the kernel only performs integer operations. Figure 9 shows the performance of our kernel depicted using the Roofline model on the NVIDIA H100 GPU. Here, we describe our analysis on the NVIDIA H100 only; the same considerations can be applied to the NVIDIA A100. Our target GPU, the NVIDIA Tesla H100 GPU, has 114 SMs available, where each SM consists of four processing blocks, called *warp schedulers*. Each warp scheduler within each block of the SM can dispatch only one instruction per cycle, correspondent to a maximum of three operation when scheduling a Multiply-Accumulate (MAC). As such, the theoretical maximum of operations/s is 114 SM × (4 Warp schedulers per SM) × (32 Instructions per Warp)× $(1\,^{\text{instruction}}/_{\text{cycle}}) \times (3\,^{\text{ops}}/_{\text{instruction}}) \times 1.170$ GHz = 51.21 TOPS. Furthermore, each processing SM contains 32 FP32, 16 FP64, and 16 INT32 cores. Therefore, the peak integer performance for the NVIDIA H100 is then $^{16}/_{32} \times 51.21 = 25.60$ INT32 TOPS[3] since 16 INT32 cores can only support 16 threads out of 32 threads in one warp. The maximum performance is upper bounded by both the theoretical INT32 peak rate and the peak memory bandwidth, defined by the blue line in Figure 9. This maximum attainable performance represents a ceiling in the Roofline model plot for the considered GPU platform and is independent of the executed algorithm. A kernel is compute-bound if limited by the hardware performance limit (blue line/light blue area), and it is memory-bound if the TOPS are limited by the memory bandwidth (purple line/purple area). To better contextualize our attained performance, we adapt the Roofline ceiling to the *KSW2* algorithm, following the same methodology described in the literature for adapting the Roofline to similar alignment algorithms [7, 35]. Indeed, given that the number of parallel elements that we can compute in parallel depends on each iteration of our target aligner (in this case *ksw2d*), we can describe the new peak with:

$$Ceiling = \frac{1}{N}\sum_{i=1}^{N}\frac{f \times N_{op,i} \times B}{\lceil (T \times B)/MAX_R \rceil}. \quad (3)$$

Equation (3) defines the new maximum performance by averaging the number of cells that GPU can compute in parallel. N indicates the total number of parallel iterations for a given algorithm, f is the GPU operating frequency, B is the number of scheduled blocks, $N_{op,i}$ indicates the number of operations that need to be computed at each iteration, T is the number of scheduled threads per block, and MAX_R indicates the number of INT32 cores available. Our implementation's overall performance behavior is shown in Figure 9. Result show the operational intensity of our kernel on the HBM memory of the GPU, indicating that our implementation is not memory bound, but that we are bound by the theoretical peak performance ceiling. Note that the optimized performance of our algorithm is very close to the adapted theoretical ceiling. Considering that the adapted ceiling does not take into account memory latency, the results of our implementation are extremely close to the maximum achievable performance. Therefore, our design represents a near-optimal implementation of the *KSW2* algorithm, and it is only limited by the computing capability of the GPU.

8 Conclusion

Our work presents the first high-performance GPU implementation of the full *KSW2* alignment suite. *KSW2* is employed in several important genomics applications, however it is particularly challenging for GPU parallelization due to its complex structure and its multiple heuristics. Our design demonstrated runtime improvements of up to 72.83×, compared with the original SIMD vectorized CPU algorithm. Additionally, results show speed-ups up to 66.00× using an NVIDIA H100 compared with the recently proposed *ksw2-fast* algorithm [11], which implements the double gap affine aligner of *KSW2* with AVX2 and AVX512 intrinsics. Our implementation also outperforms the state-of-the-art *ksw2z* FPGA implementation by up to a factor of 156.37× using a single H100 GPU. Finally, our integration resulted in performance improvements up to 8.51× and 8.03× against *minimap2* and *mm2-fast*, respectively using a single GPU, proving the effectiveness of our design on read-world applications. Furthermore, our Roofline analysis demonstrates that our design methodology results in near-optimal performance. Our overall results show that our optimized kernel is flexible, efficient, and can be easily integrated into a wide set of genomic applications performing long-read alignment.

[3] https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper

References

[1] Stephen F Altschul, Thomas L Madden, Alejandro A Schäffer, Jinghui Zhang, Zheng Zhang, Webb Miller, and David J Lipman. 1997. Gapped BLAST and PSI-BLAST: a new generation of protein database search programs. *Nucleic acids research* 25, 17 (1997), 3389–3402.

[2] Mia Yang Ang, Teck Yew Low, Pey Yee Lee, Wan Fahmi Wan Mohamad Nazarie, Victor Guryev, and Rahman Jamal. 2019. Proteogenomics: from next-generation sequencing (NGS) and mass spectrometry-based proteomics to precision medicine. *Clinica chimica acta* 498 (2019), 38–46.

[3] Christiam Camacho, George Coulouris, Vahram Avagyan, Ning Ma, Jason Papadopoulos, Kevin Bealer, and Thomas L Madden. 2009. BLAST+: architecture and applications. *BMC bioinformatics* 10, 1 (2009), 1–9.

[4] Alejandro Chacón, Santiago Marco-Sola, Antonio Espinosa, Paolo Ribeca, and Juan Carlos Moure. 2014. Thread-cooperative, bit-parallel computation of levenshtein distance on GPU. In *Proceedings of the 28th ACM international conference on Supercomputing*. 103–112.

[5] Nan Ding and Samuel Williams. 2019. An Instruction Roofline Model for GPUs. *2019 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)* (2019).

[6] Can Firtina, Jisung Park, Mohammed Alser, Jeremie S Kim, Damla Senol Cali, Taha Shahroodi, Nika Mansouri Ghiasi, Gagandeep Singh, Konstantinos Kanellopoulos, Can Alkan, et al. 2023. BLEND: a fast, memory-efficient and accurate mechanism to find fuzzy seed matches in genome analysis. *NAR Genomics and Bioinformatics* 5, 1 (2023), lqad004.

[7] Giulia Gerometta, Alberto Zeni, and Marco D Santambrogio. 2023. TSUNAMI: A GPU implementation of the WFA algorithm. In *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 150–161.

[8] Sneha D Goenka, Yatish Turakhia, Benedict Paten, and Mark Horowitz. 2020. SegAlign: A scalable GPU-based whole genome aligner. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–13.

[9] Osamu Gotoh. 1982. An improved algorithm for matching biological sequences. *Journal of molecular biology* 162, 3 (1982), 705–708.

[10] Rehan Hameed, Wajahat Qadeer, Megan Wachs, Omid Azizi, Alex Solomatnikov, Benjamin C Lee, Stephen Richardson, Christos Kozyrakis, and Mark Horowitz. 2010. Understanding sources of inefficiency in general-purpose chips. In *Proceedings of the 37th annual international symposium on Computer architecture*. 37–47.

[11] Saurabh Kalikar, Chirag Jain, Vasimuddin Md, and Sanchit Misra. 2022. Accelerating long-read analysis on modern CPUs. *bioRxiv* (2022), 2021–07.

[12] Ben Langmead and Steven L Salzberg. 2012. Fast gapped-read alignment with Bowtie 2. *Nature methods* 9, 4 (2012), 357.

[13] Heng Li. 2013. Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. *arXiv preprint arXiv:1303.3997* (2013).

[14] Heng Li. 2018. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics* 34, 18 (2018), 3094–3100.

[15] Chi-Man Liu, Thomas Wong, Edward Wu, Ruibang Luo, Siu-Ming Yiu, Yingrui Li, Bingqiang Wang, Chang Yu, Xiaowen Chu, Kaiyong Zhao, et al. 2012. SOAP3: ultra-fast GPU-based parallel alignment tool for short reads. *Bioinformatics* 28, 6 (2012), 878–879.

[16] Yongchao Liu, Tuan-Tu Tran, Felix Lauenroth, and Bertil Schmidt. 2014. SWAPHI-LS: Smith-Waterman algorithm on Xeon Phi coprocessors for long DNA sequences. In *2014 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 257–265.

[17] DR Mani, Karsten Krug, Bing Zhang, Shankha Satpathy, Karl R Clauser, Li Ding, Matthew Ellis, Michael A Gillette, and Steven A Carr. 2022. Cancer proteogenomics: current impact and future prospects. *Nature Reviews Cancer* 22, 5 (2022), 298–313.

[18] Saul B Needleman and Christian D Wunsch. 1970. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of molecular biology* 48, 3 (1970), 443–453.

[19] Torbjørn Rognes and Erling Seeberg. 2000. Six-fold speed-up of Smith–Waterman sequence database searches using parallel processing on common microprocessors. *Bioinformatics* 16, 8 (2000), 699–706.

[20] Harisankar Sadasivan, Milos Maric, Eric Dawson, Vishanth Iyer, Johnny Israeli, and Satish Narayanasamy. 2023. Accelerating Minimap2 for accurate long read alignment on GPUs. *Journal of biotechnology and biomedicine* 6, 1 (2023), 13.

[21] Eric E Schadt, Steve Turner, and Andrew Kasarskis. 2010. A window into third-generation sequencing. *Human molecular genetics* 19, R2 (2010), R227–R240.

[22] Gloria M Sheynkman, Michael R Shortreed, Anthony J Cesnik, and Lloyd M Smith. 2016. Proteogenomics: integrating next-generation sequencing and mass spectrometry to characterize human proteomic variation. *Annual review of analytical chemistry* 9 (2016), 521–545.

[23] Temple F Smith, Michael S Waterman, et al. 1981. Identification of common molecular subsequences. *Journal of molecular biology* 147, 1 (1981), 195–197.

[24] Zachary D Stephens, Skylar Y Lee, Faraz Faghri, Roy H Campbell, Chengxiang Zhai, Miles J Efron, Ravishankar Iyer, Michael C Schatz, Saurabh Sinha, and Gene E Robinson. 2015. Big data: astronomical or genomical? *PLoS biology* 13, 7 (2015), e1002195.

[25] Pawel Suwinski, ChuangKee Ong, Maurice HT Ling, Yang Ming Poh, Asif M Khan, and Hui San Ong. 2019. Advancing personalized medicine through the application of whole exome sequencing and big data analytics. *Frontiers in genetics* 10 (2019), 49.

[26] Hajime Suzuki and Masahiro Kasahara. 2018. Introducing difference recurrence relations for faster semi-global alignment of long sequences. *BMC bioinformatics* 19, 1 (2018), 45.

[27] Carolina Teng. 2022. Accelerating the alignment phase of Minimap2 genome assembly algorithm Using GACT-X in a commercial Cloud FPGA machine.

[28] Carolina Teng, Renan W Achjian, Caio C Braga, Marcelo K Zuffo, and Wang J Chau. 2021. Accelerating the base-level alignment step of DNA assembling in Minimap2 Algorithm using FPGA. In *2021 IEEE 12th Latin America Symposium on Circuits and System (LASCAS)*. IEEE, 1–4.

[29] Yatish Turakhia, Gill Bejerano, and William J Dally. 2018. Darwin: A genomics co-processor provides up to 15,000 x acceleration on long read assembly. In *ACM SIGPLAN Notices*, Vol. 53. ACM, 199–213.

[30] Eric S Tvedte, Mark Gasser, Benjamin C Sparklin, Jane Michalski, Carl E Hjelmen, J Spencer Johnston, Xuechu Zhao, Robin Bromley, Luke J Tallon, Lisa Sadzewicz, et al. 2021. Comparison of long-read sequencing technologies in interrogating bacteria and fly genomes. *G3* 11, 6 (2021), jkab083.

[31] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.

[32] Qisheng Xu, Yong Dou, and Yanjie Sun. 2022. Accelerating minimap2 for long-read sequencing on NUMA multi-core CPU. In *Proceedings of the 5th International Conference on Computer Science and Software Engineering*. 152–158.

[33] Alberto Zeni, Guido Walter Di Donato, Alessia Della Valle, Filippo Carloni, and Marco D Santambrogio. 2023. On the Genome Sequence Alignment FPGA Acceleration via KSW2z. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.

[34] Alberto Zeni, Guido Walter Di Donato, Lorenzo Di Tucci, Marco Rabozzi, and Marco D Santambrogio. 2021. The Importance of Being X-Drop: High Performance Genome Alignment on Reconfigurable Hardware. In *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 133–141.

[35] Alberto Zeni, Giulia Guidi, Marquita Ellis, Nan Ding, Marco D Santambrogio, Steven Hofmeyr, Aydın Buluç, Leonid Oliker, and Katherine Yelick. 2020. Logan: High-performance gpu-based x-drop long-read alignment. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 462–471.

[36] Anbo Zhou, Timothy Lin, and Jinchuan Xing. 2019. Evaluating nanopore sequencing data processing pipelines for structural variation identification. *Genome biology* 20 (2019), 1–13.